Activity – Concept Check
Date: 18.09.2026
1. An organization is granted the block 16.0.0.0/8. The administrator wants to create 500 fixed
length subnets.
a. Find the first and the last address in the first subnet.
b. Find the first and the last address in the last subnet.
c. Find the number of addresses in each subnet.
d. Find the subnet prefix.
2. Convert the following 32-bit IPv4 address from binary notation to dotted-decimal notation and

Show more

Absolutely. Let's solve **all 10 Concept Check questions step-by-step**, in an exam-friendly way. I'll also point out the **key formula/concept** used in each.

1. Block 16.0.0.0/8 → 500 Fixed-Length Subnets

Given:

- Network = **16.0.0.0/8**

- Required subnets = **500**

Step 1: Find number of subnet bits

We need:

$$2^n \geq 500$$

$$2^8 = 256 < 500$$

$$2^9 = 512 \geq 500$$

Therefore, we borrow **9 bits** from the host portion.

Original prefix:

$$/8$$

New prefix:

$$8 + 9 = \ /17$$

So we get **512 subnets**, of which 500 are required.

Step 2: Number of addresses per subnet

A `/17` network leaves:

$$32 - 17 = 15$$

host bits.

Therefore:

$$2^{15} = 32768$$

addresses per subnet.

(c) Answer

$$32768 \text{ addresses/subnet}$$

First subnet

Network:

$$16.0.0.0/17$$

A `/17` mask is:

```
11111111.11111111.10000000.00000000
255.255.128.0
```

The first subnet covers:

```
16.0.0.0 → 16.0.127.255
```

(a) Answer

First address:

$$16.0.0.0$$

Last address:

$$16.0.127.255$$

Last subnet

There are 512 possible subnets.

Subnet increment is **128 in the third octet**:

```
16.0.0.0
16.0.128.0
16.1.0.0
16.1.128.0
...
```

The last subnet is:

$$16.255.128.0/17$$

It extends to:

$$16.255.255.255$$

(b) Answer

First:

$$16.255.128.0$$

Last:

$$16.255.255.255$$

Final answers

Part	Answer
a	**16.0.0.0 – 16.0.127.255**
b	**16.255.128.0 – 16.255.255.255**
c	**32,768 addresses/subnet**
d	**/17**

2. Binary ↔ Dotted Decimal

Remember the positional values of an 8-bit octet:

```
128 64 32 16 8 4 2 1
```

(a)

```
11000000 10101000 00001010 01100101
```

First octet

```
11000000
= 128 + 64
= 192
```

Second

```
10101000
= 128 + 32 + 8
= 168
```

Third

```
00001010
= 8 + 2
= 10
```

Fourth

```
01100101
= 64 + 32 + 4 + 1
= 101
```

Therefore:

192.168.10.101

(b)

```
10101100 00010000 00000101 11001010
```

```
10101100 = 128+32+8 = 168

00010000 = 16

00000101 = 4+1 = 5

11001010 = 128+64+8+2 = 202
```

Therefore:

168.16.5.202

(c)

Convert:

$$168.110.121.12$$

168

$$128 + 32 + 8 = 168$$

```
10101000
```

110

$$64 + 32 + 8 + 4 + 2 = 110$$

```
01101110
```

121

$$64 + 32 + 16 + 8 + 1 = 121$$

```
01111001
```

12

$$8 + 4 = 12$$

```
00001100
```

Therefore:

$$10101000\ 01101110\ 01111001\ 00001100$$

(d)

$$126.92.56.110$$

```
126 = 01111110
92  = 01011100
56  = 00111000
110 = 01101110
```

Therefore:

$$01111110\ 01011100\ 00111000\ 01101110$$

3. TCP Sequence Numbers

Given:

- First byte = **15,000**
- Total data = 9,000 bytes
- 5 segments:

Segment	Size
1	1500 bytes
2	1500 bytes
3	2000 bytes
4	2000 bytes
5	2000 bytes

Important TCP rule

The sequence number represents the **first byte of data in that segment**.

Segment 1

Starts at:

15000

It carries bytes:

```
15000 – 16499
```

Next sequence number:

$$15000 + 1500 = 16500$$

Segment 2

Starts at:

$$16500$$

Next:

$$16500 + 1500 = 18000$$

Segment 3

Starts at:

$$18000$$

Next:

$$18000 + 2000 = 20000$$

Segment 4

Starts at:

$$20000$$

Next:

$$20000 + 2000 = 22000$$

Segment 5

Starts at:

$$22000$$

Final answer

Segment	Data size	Sequence number
1	1500	**15,000**
2	1500	**16,500**
3	2000	**18,000**
4	2000	**20,000**
5	2000	**22,000**

4. Find the Class of IP Addresses

Remember:

Class	First octet
A	1–126
B	128–191
C	192–223
D	224–239
E	240–255

(a) 208.34.54.12

First octet = 208

```
192–223 → Class C
```

Class C

(b) 238.34.2.1

238 lies between 224–239.

Class D

Class D is used for **multicast**.

(c) 114.34.2.8

114 lies between 1–126.

Class A

(d)

```
10101111 11000000 11110000 00011101
```

First octet:

```
10101111
= 128 + 32 + 8 + 4 + 2 + 1
= 175
```

175 → Class B.

Class B

(e)

```
11011111 10110000 00011111 01011101
```

First octet:

```
11011111 = 223
```

223 → Class C.

Class C

(f)

```
11101111 11110111 11000111 00011101
```

First octet:

```
11101111 = 239
```

239 → Class D.

Class D

Final

Address	Class
208.34.54.12	C
238.34.2.1	D
114.34.2.8	A
10101111...	B
11011111...	C
11101111...	D

5. Find NetID and HostID

For **classful addressing**:

Class	NetID	HostID
A	1 octet	3 octets

Class	NetID	HostID
B	2 octets	2 octets
C	3 octets	1 octet

(a) 114.34.2.8

114 → Class A.

Therefore:

```
NetID  = 114
HostID = 34.2.8
```

$$\text{NetID} = 114$$

$$\text{HostID} = 34.2.8$$

(b) 132.56.8.6

132 → Class B.

```
NetID  = 132.56
HostID = 8.6
```

$$\text{NetID} = 132.56$$

$$\text{HostID} = 8.6$$

(c) 208.34.54.12

208 → Class C.

```
NetID  = 208.34.54
HostID = 12
```

$$\text{NetID} = 208.34.54$$

$$\text{HostID} = 12$$

6. IPv4 Multicast → Ethernet Multicast

This is a very important concept.

IPv4 multicast addresses:

$$224.0.0.0 \text{ to } 239.255.255.255$$

are mapped to Ethernet multicast addresses beginning with:

$$01:00:5E$$

Only the **lower 23 bits** of the IPv4 multicast address are mapped.

Formula:

```
Ethernet = 01:00:5E + lower 23 bits of IPv4 multicast address
```

(a) 224.18.72.8

IPv4:

```
224.18.72.8
```

The first 8 bits are discarded except for the mapping prefix.

Take lower 23 bits:

```
18      = 00010010
72      = 01001000
8       = 00001000
```

Result:

$$01:00:5E:12:48:08$$

(b) 235.18.72.8

The lower 23 bits are again:

```
18.72.8
```

Therefore:

$$01:00:5E:12:48:08$$

Notice something important:

(a) and (b) produce the same Ethernet address.

(c) 237.18.6.88

Lower 23 bits:

```
18.6.88
```

Therefore:

$$01:00:5E:12:06:58$$

(d) 224.88.12.8

Lower 23 bits:

```
88.12.8
```

Therefore:

$$01:00:5E:58:0C:08$$

Final

IPv4 multicast	Ethernet multicast
224.18.72.8	**01:00:5E:12:48:08**
235.18.72.8	**01:00:5E:12:48:08**
237.18.6.88	**01:00:5E:12:06:58**
224.88.12.8	**01:00:5E:58:0C:08**

Therefore:

$$\text{2 addresses specify the same Ethernet address}$$

Specifically, **(a) and (b)**.

7. ARP Request Packet

Given router:

Router IP

$$125.45.23.12$$

Router MAC

$$23:45:AB:4F:67:CD$$

Destination host:

$$125.11.78.10$$

No subnetting.

What does ARP do?

The router knows:

```
Destination IP = 125.11.78.10
```

but doesn't know its MAC address.

So it broadcasts an ARP Request:

"Who has IP 125.11.78.10?"

The ARP request contains:

Field	Value
Hardware Type	Ethernet
Protocol Type	IPv4
Hardware Address Length	6
Protocol Address Length	4
Operation	**ARP Request (1)**
Sender MAC	**23:45:AB:4F:67:CD**
Sender IP	**125.45.23.12**
Target MAC	**00:00:00:00:00:00**
Target IP	**125.11.78.10**

Ethernet frame

Because it is an ARP request, it is broadcast:

Destination MAC:

$$FF:FF:FF:FF:FF:FF$$

Source MAC:

$$23:45:AB:4F:67:CD$$

Key exam point

The target MAC is unknown, so:

$$00:00:00:00:00:00$$

8. IGMP Query

Host IP:

$$222.5.7.19$$

Routing table contains:

```
227.4.3.7
229.45.6.23
```

Both are multicast addresses.

The host receives an **IGMP Query**.

What should it do?

IGMP is used to manage **multicast group membership**.

The host sees that it has memberships in **two multicast groups**:

1. 227.4.3.7
2. 229.45.6.23

Therefore, it needs to respond for the multicast groups it belongs to.

It sends:

2 IGMP Membership Reports

One for:

```
227.4.3.7
```

and one for:

```
229.45.6.23
```

Answer

Send 2 IGMP Membership Reports

One for each multicast group.

Exam note: In actual IGMP behavior, hosts use randomized response timers to avoid everyone replying simultaneously, so multiple hosts may suppress redundant reports. For this textbook question, the intended answer is **two membership reports**, one for each listed group.

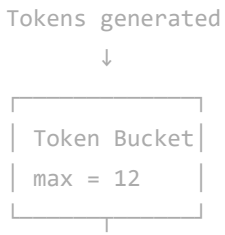
9. Token Bucket

Given:

- Normal traffic = **3 Mbps**
- Burst traffic = **7 Mbps**
- Bucket capacity = **12 tokens**
- Token generation rate = **3 tokens/sec**
- 1 token = 1 packet

(i) How does Token Bucket decide whether packet can be transmitted?

The basic mechanism is:





For every packet:

If at least one token exists:

- Remove one token.
- Transmit the packet immediately.

If no token exists:

- The packet must **wait** until a token becomes available, or may be dropped depending on the implementation.

Tokens are generated at:

$$3 \text{ tokens/sec}$$

So under sustained traffic, the long-term transmission rate is controlled to approximately the token generation rate.

(ii) How can burst traffic exceed the normal rate?

This is the major advantage of Token Bucket.

Suppose the network is idle for some time.

Tokens accumulate.

Maximum:

$$12 \text{ tokens}$$

Now suppose a sudden burst occurs.

Because 12 tokens are already available, up to:

$$12 \text{ packets}$$

can be transmitted immediately, assuming packet transmission capacity permits it.

Therefore, the sender can temporarily transmit at a rate **higher than 3 packets/sec**.

After the stored tokens are consumed, packets must follow the token generation rate:

$$3 \text{ tokens/sec}$$

In simple words:

Token bucket allows bursts but controls the long-term average rate.

That's the sentence I'd remember for the exam.

10. 68,000-byte Packet Using One UDP Datagram

This question is about **UDP's maximum user datagram size and IP fragmentation**.

Given:

$$68,000 \text{ bytes}$$

UDP has a maximum theoretical user datagram size of:

$$65,507 \text{ bytes}$$

because IPv4 has a maximum packet size of 65,535 bytes:

$$65,535 - 20 \text{ (IP header)} - 8 \text{ (UDP header)}$$

$$= 65,507$$

Therefore, **68,000 bytes cannot fit into a single unfragmented IPv4 packet**.

But the question specifically says:

> "using only one UDP user datagram"

That is possible because **IP fragmentation occurs below UDP**.

Step 1: UDP creates ONE datagram

UDP sees:

```
Application data = 68,000 bytes
```

UDP adds its 8-byte header.

So UDP datagram size:

$$68,000 + 8$$

$$= 68,008 \text{ bytes}$$

UDP considers this **one user datagram**.

Step 2: IP receives the UDP datagram

IPv4 maximum packet size is:

$$65,535$$

The 68,008-byte UDP datagram is too large to fit into one IPv4 packet.

Therefore, IP fragments it into multiple IP fragments.

Important distinction

```
Application
    ↓
ONE UDP datagram
    ↓
IP fragmentation
    ↓
Multiple IP fragments
    ↓
Network
```

At the receiver:

```
Multiple IP fragments
        ↓
IP reassembly
        ↓
ONE UDP datagram
        ↓
UDP
        ↓
Application
```

Therefore:

> One UDP user datagram can be fragmented into multiple IP packets

UDP itself does **not** perform the fragmentation.

⭐ Quick Revision Sheet

Q	Key answer
1	500 subnets → borrow 9 bits → **/17**, 32,768 addresses/subnet
2	Binary ↔ decimal using **128 64 32 16 8 4 2 1**
3	TCP sequence number = **first byte number**
4	A: 1–126, B: 128–191, C: 192–223, D: 224–239, E: 240–255
5	A = 1 octet NetID, B = 2, C = 3
6	IPv4 multicast → `01:00:5E` + lower 23 bits
7	ARP Request → target MAC = `00:00:00:00:00:00` , Ethernet destination = broadcast
8	**2 IGMP Membership Reports**
9	Token bucket permits **bursts using accumulated tokens**, controls long-term rate
10	**One UDP datagram → multiple IP fragments**

Share •••

Most important formulas/concepts to memorize

$$2^n \geq \text{required subnets}$$

$$\text{Addresses/subnet} = 2^{32-\text{prefix}}$$

$$\text{TCP next sequence number} = \text{current sequence number} + \text{segment data byt}$$

$$\text{IPv4 multicast Ethernet} = 01:00:5E + \text{lower 23 bits}$$

$$\text{ARP Request} = \text{"Who has this IP?"}$$

$$\text{Token Bucket} = \text{permits bursts} + \text{controls average rate}$$

$$\text{UDP datagram can be fragmented by IP}$$

ChatGPT can make mistakes. Check important info.